



**EDUCACIÓN**  
SECRETARÍA DE EDUCACIÓN PÚBLICA



**TECNOLÓGICO NACIONAL DE MÉXICO  
INSTITUTO TECNOLÓGICO DE TIJUANA**

**SUBDIRECCIÓN ACADÉMICA  
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN**

**SEMESTRE:**  
Enero-Junio

**CARRERA:**  
Ingeniería en Sistemas Computacionales

**MATERIA:**  
Patrones de Diseño

**TÍTULO ACTIVIDAD:**  
Patrón de Diseño State

**NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:**

Chávez Moreno Roberto 22210292

**NOMBRE DEL MAESTRO:**

Maribel Guerrero Luis

**FECHA:**

Miércoles 6 mayo 2026

## Introducción

El patrón State es un patrón de diseño de comportamiento que permite a un objeto modificar su comportamiento cuando su estado interno cambia, haciendo que parezca que ha cambiado de clase. Este patrón está compuesto por tres elementos: el Contexto (objeto principal), la interfaz Estado (que define los métodos comunes) y los Estados Concretos (clases que implementan comportamientos específicos).

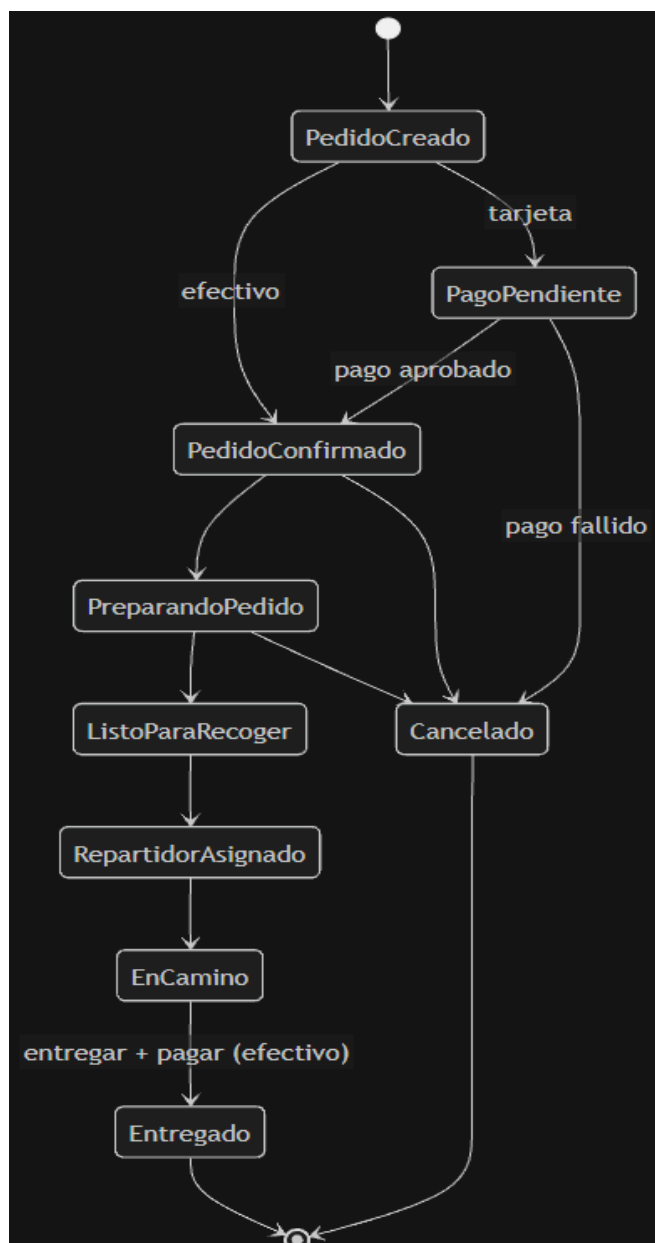
En las aplicaciones de delivery, un pedido atraviesa múltiples etapas: creado, confirmado, preparándose, en camino y entregado. Gestionar estos estados con condicionales tradicionales (if-else o switch) genera código difícil de mantener y propenso a errores.

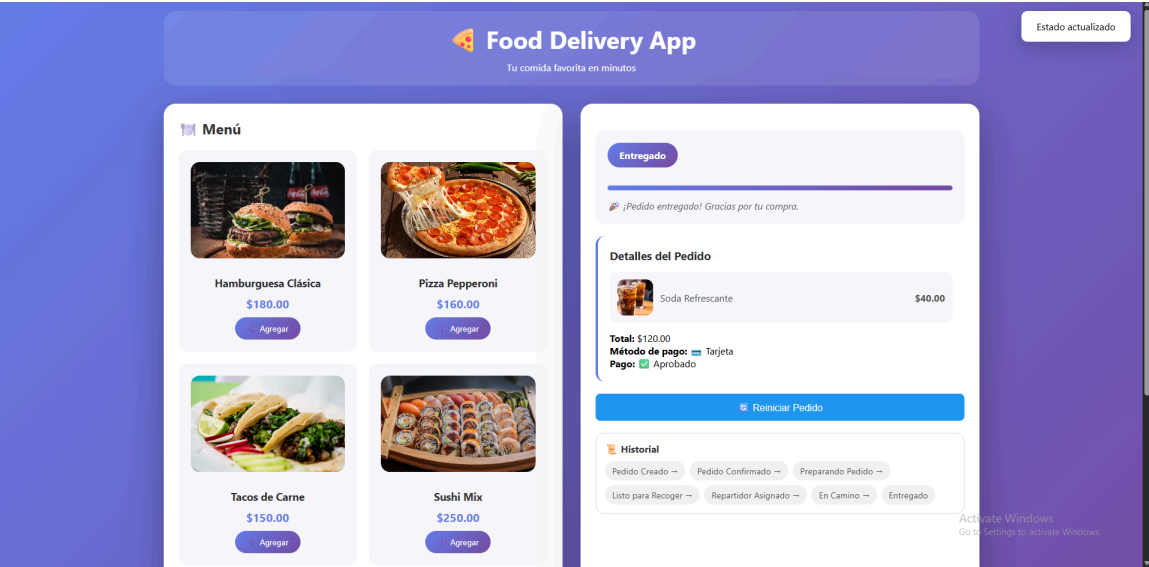
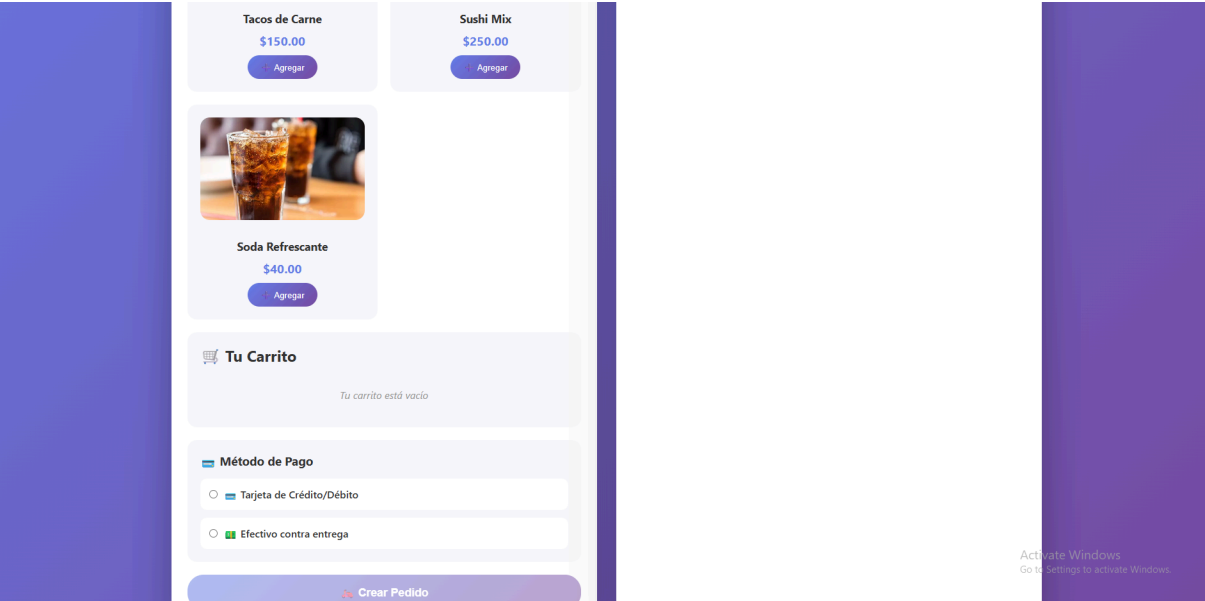
Este proyecto implementa el patrón State para gestionar el ciclo de vida de pedidos de comida. Cada estado del pedido es una clase independiente que define dos acciones clave: avanzarEstado (transición al siguiente estado) y cancelarPedido (anulación si está permitido). El contexto (Pedido) delega todas las operaciones al estado actual, eliminando por completo los condicionales centralizados.

## Definición de Estados

| Estado                        | Descripción                                                                                |
|-------------------------------|--------------------------------------------------------------------------------------------|
| <b>1. Pedido Creado</b>       | Estado inicial donde el cliente realiza el pedido, pero el restaurante aún no lo confirma. |
| <b>2. Pedido Confirmado</b>   | El restaurante acepta el pedido y comienza a procesarlo.                                   |
| <b>3. Preparando Pedido</b>   | La comida se encuentra en proceso de preparación.                                          |
| <b>4. Listo para Recoger</b>  | El pedido ya está listo y espera asignación o recogida por un repartidor.                  |
| <b>5. Repartidor Asignado</b> | Un repartidor ha aceptado entregar el pedido.                                              |

|                     |                                                                                                               |
|---------------------|---------------------------------------------------------------------------------------------------------------|
| <b>6. En Camino</b> | El repartidor se dirige hacia el cliente.                                                                     |
| <b>7. Entregado</b> | Estado final exitoso donde el pedido llega al cliente.                                                        |
| <b>8. Cancelado</b> | Estado final alternativo al que solo se puede llegar desde: Pedido Creado Pedido confirmado Preparando pedido |





```

models > JS EstadoPedido.js > ...
// Clase base abstracta para todos los estados del pedido
// Define el contrato que deben seguir todos los estados
class EstadoPedido {
  constructor(pedido) {
    this.pedido = pedido;
  }

  // Método abstracto para avanzar al siguiente estado
  avanzarEstado() {
    throw new Error(`Método avanzarEstado debe ser implementado`);
  }

  // Método abstracto para cancelar el pedido
  cancelarPedido() {
    throw new Error(`Método cancelarPedido debe ser implementado`);
  }

  // Método abstracto para obtener el nombre del estado
  getNombre() {
    throw new Error(`Método getNombre debe ser implementado`);
  }

  // Método abstracto para obtener la descripción del estado
  getDescripcion() {
    throw new Error(`Método getDescripcion debe ser implementado`);
  }

  // Por defecto, los pedidos no se pueden cancelar
  puedeCancelar() {
    return false;
  }
}

export default EstadoPedido;

```

## Conclusión

La implementación del patrón state en esta app estilo uber eats fue una muy buena solución para manejar todo el proceso de un pedido sin usar muchos if o switch. Cada estado es como una clase que sabe qué hacer, lo que hace que el código sea más ordenado, claro y fácil de cambiar.